alltoall/bruck

An AgentMPI collective scaling analysis

Abstract

Bruck's algorithm is the one `alltoall` schedule that changes the asymptotics rather than the ordering. By rotating each rank's send buffer locally and then exchanging doubling-distance blocks with a store-and-forward step, it completes the full p-way exchange in $\lceil \log_2 p \rceil$ rounds and $p\lceil \log_2 p \rceil$ messages instead of $p-1$ rounds and $p(p-1)$ messages — 160 messages in 5 rounds at $p = 32$, against 992 in 31 for the other two algorithms. Both counts hold at all twenty-one measured sizes with no disagreement between the closed form, the self-report and the log, and the round count steps exactly at 2, 4, 8, 16 and 32 as $\lceil \log_2 p \rceil$ requires. The price is volume: the intermediate messages carry other ranks' data, and the sweep measured 7,680 payload tokens at $p = 32$ against 2,976 for `pairwise`, a factor of 2.58. The family view also exposes a real inaccuracy in the cost model that no single run could show. The tabulated volume term $p\lceil \log_2 p \rceil np/2$ is exact only when p is a power of two; at every other size it over-predicts, by 50 % at $p = 3$ and $p = 5$, 33 % at $p = 10$ and 25 % at $p = 20$, because the number of block indices carrying a given bit is $p/2$ only when p is a power of two. The implementation is right and the formula is the approximation.

1 What this family is

The `alltoall` collective under the `bruck` algorithm, measured across 21 runs at 13 distinct process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.014–0.302 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

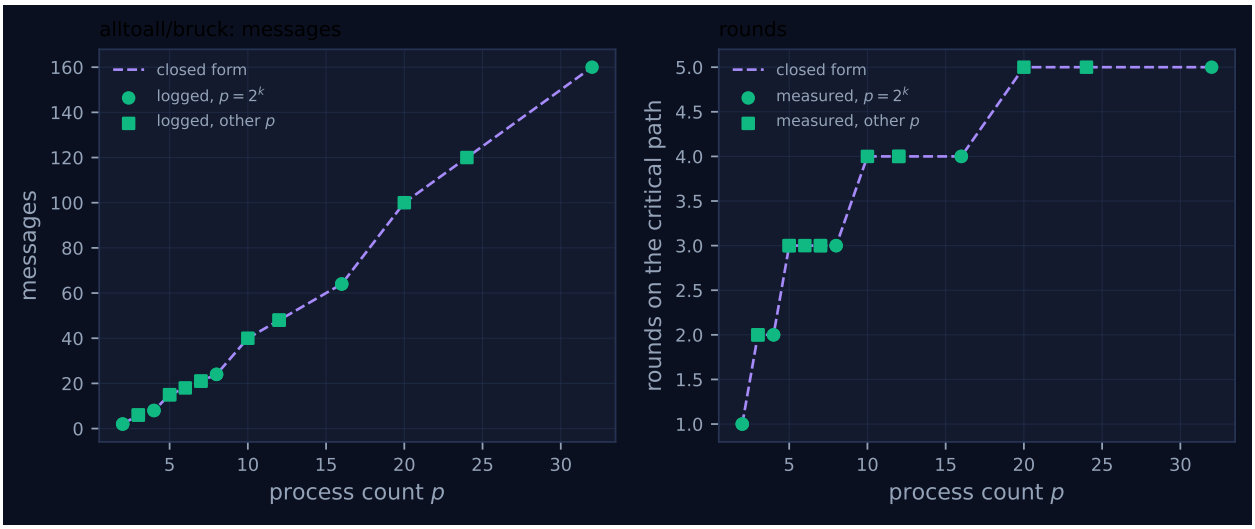

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	1	1	0.014	✓	✓
2	`mb-smoke`	2	2	2	1	1	0.014	✓	✓
3	`mb-free`	6	6	6	2	2	0.016	✓	✓
3	`mb-smoke`	6	6	6	2	2	0.018	✓	✓
4	`mb-free`	8	8	8	2	2	0.030	✓	✓
4	`mb-smoke`	8	8	8	2	2	0.028	✓	✓
5	`mb-free`	15	15	15	3	3	0.064	✓	✓
5	`mb-smoke`	15	15	15	3	3	0.042	✓	✓
6	`mb-free`	18	18	18	3	3	0.049	✓	✓
7	`mb-free`	21	21	21	3	3	0.054	✓	✓
7	`mb-smoke`	21	21	21	3	3	0.043	✓	✓
8	`mb-free`	24	24	24	3	3	0.042	✓	✓
8	`mb-smoke`	24	24	24	3	3	0.059	✓	✓
10	`mb-free`	40	40	40	4	4	0.066	✓	✓
12	`mb-free`	48	48	48	4	4	0.080	✓	✓
12	`mb-smoke`	48	48	48	4	4	0.101	✓	✓
16	`mb-free`	64	64	64	4	4	0.120	✓	✓
16	`mb-smoke`	64	64	64	4	4	0.147	✓	✓
20	`mb-free`	100	100	100	5	5	0.217	✓	✓
24	`mb-free`	120	120	120	5	5	0.302	✓	✓
32	`mb-free`	160	160	160	5	5	0.256	✓	✓

4 How the algorithm scales

The implementation is worth reading closely, because the cost falls straight out of it. Each rank first rotates its own send buffer, `rot[i] = payloads[(r + i) % p]`, so that position i holds the item destined for the rank i steps ahead. Then for $k = 0 \dots \lceil \log_2 p \rceil - 1$ it computes dist $= 2^k$, selects

every buffer index whose k-th bit is set, and exchanges that entire set of items with its neighbours at distance $\pm 2^k$ in one `sendrecv`. Finally it un-rotates, placing `rot[i]` at `out[(r - i) % p]`.

Two consequences follow. First, the message count is one exchange per rank per round, so $p\lceil \log_2 p \rceil$ in total and $\lceil \log_2 p \rceil$ rounds on the critical path — exactly `FORMULAS["alltoall","bruck"]`. Second, and this is the part the message count hides, each message carries roughly half the buffer rather than one item: the set $\{i < p : i \wedge 2^k\}$ has about $p/2$ members, so each rank forwards about $np/2$ tokens per round and $np\lceil \log_2 p \rceil/2$ overall. Data belonging to rank j therefore passes through intermediate ranks on its way to rank j; that is what "store and forward" means here, and it is the entire cost of the logarithmic depth.

Figure 1 shows both effects and the contrast with the family's siblings is stark. The message panel is $p\lceil \log_2 p \rceil$, a staircase that flattens between powers of two: 24 messages at $p = 8$, 64 at $p = 16$, 160 at $p = 32$, against $p(p-1) = 56$, 240 and 992. The round panel steps only at 2, 4, 8, 16 and 32, holding at 3 across $p = 5, 6, 7, 8$ and at 4 across $p = 10, 12, 16$, which is what $\lceil \log_2 p \rceil$ does and where the closed form's steps should be. The measured round count matches at every one of the twenty-one rows.

The saving is real in the measurements as well as in the counts. At $p = 32$ the whole collective spans 0.256 s against 0.959 s for `linear` and 1.135 s for `pairwise`, and the aggregate time ranks spent blocked in a receive was 4.59 rank-seconds against 250.6 and 30.6 — a 55-fold and 6.7-fold reduction respectively, achieved by moving 6.2 times fewer messages through the same fabric. The aggregate blocking grows as $p^{2.14}$ ($R^2 = 0.984$), the gentlest exponent of the three schedules. The viewer screenshot for `mb-free__coll-alltoall-bruck-32` shows the structure directly: 418 events against 2,082 for `linear` at the same population, every lane carrying about ten glyphs in five loose clusters, no lane empty and no lane privileged. The five clusters are the five rounds, loosely aligned because ranks proceed independently between exchanges.

5 Powers of two and everything else

This is the family where the non-power-of-two path matters most, and it needs separating into two questions: does the implementation behave, and does the closed form describe it?

The implementation behaves. Message count and round count agree with the closed form at all twenty-one sizes, including 15 messages in 3 rounds at $p = 5$, 18 in 3 at $p = 6$, 21 in 3 at $p = 7$, 40 in 4 at $p = 10$, 100 in 5 at $p = 20$ and 120 in 5 at $p = 24$. No red crosses appear in Figure 1, so the collective's self-reported `messages_sent` equalled the fabric's `msg.send` count everywhere, and the eight sizes measured in both `mb-free` and `mb-smoke` agree on every integer. That is not a trivial result for Bruck: the classic algorithm is stated for powers of two and its non-power-of-two variants are where index arithmetic goes wrong. Here the rotation makes the algorithm size-agnostic, since `dist` is compared against buffer indices in $[0, p)$ rather than against a padded power-of-two range, and the un-rotation is modulo p.

The closed form does not describe it exactly, and this is the defect the family view exposes. The volume term in `FORMULAS` is $p\lceil \log_2 p \rceil \cdot n \cdot p/2$, which assumes each round's index set holds exactly $p/2$ members. The true count is $\sum_k |\{i < p : i \wedge 2^k\}|$ per rank, and that equals $p\lceil \log_2 p \rceil/2$ only when p is a power of two. Computing both over the measured grid and comparing against the volume the collective reported gives:

p	reported (tokens)	exact $\times n$	closed form $\times n$	ratio
2	6	6	6	1.000
3	18	18	27	0.667
4	48	48	48	1.000
5	75	75	112.5	0.667
6	126	126	162	0.778
7	189	189	220.5	0.857
8	288	288	288	1.000
10	450	450	600	0.750
12	720	720	864	0.833
16	1536	1536	1536	1.000
20	2400	2400	3000	0.800
24	3744	3744	4320	0.867
32	7680	7680	7680	1.000

The reported volume equals the exact combinatorial count at every size ($n = 3$ tokens for the benchmark's `"{rank}->{j}"` payload), and equals the closed form at exactly the five powers of two. Everywhere else the closed form over-predicts, worst at $p = 3$ and $p = 5$ where it is 1.5 times the truth. The pattern — exact at $2, 4, 8, 16, 32$ and a systematic excess at every other size — is the same signature the repository already documents for recursive-doubling allreduce, and it is invisible in any single run for the same reason.

Two things distinguish it from that earlier defect and both should be said plainly. It is in the *volume* term rather than the message count, so the generated table above does not check it: the table validates `logged` against `pred.` for messages and `rounds` against `pred.` for rounds, and volume appears in neither column. And it errs conservatively — the model over-states the cost, so a harness that used it would over-provision rather than under-provision, which is the opposite of the allreduce case where the count made the collective look cheaper than it was. It is still worth fixing, because `predict` feeds `price_usd` and a 50 % over-estimate of token volume at $p = 5$ is a 50 % over-estimate of the bill.

6 When to choose this algorithm

`bruck` is the right choice when α dominates, which for agent ranks is most of the time, and the sweep quantifies both sides of the trade at $p = 32$: 5 rounds against 31, 160 messages against 992, and 7,680 payload tokens against 2,976. Running that through `cost.predict` with the default parameters at $n = 4{,}096$ tokens per destination gives 2.58 s of critical path against `pairwise`'s 15.97 s, and 10.49 M tokens of volume against 4.06 M — so 6.2 times faster and 2.6 times more expensive, or $31.46 against $12.19 at output prices. The crossover is therefore not about p; the round advantage grows with p and the volume penalty grows with p too, at $\log_2 p$ versus constant. It is about what a harness is short of. If wall time is the constraint, take `bruck`; if the token bill or the context budget is, take `pairwise`.

There is a third consideration that neither the counts nor the cost model expresses, and for agent ranks it may be the decisive one. `bruck`'s intermediate messages contain other ranks' artefacts. In the sweep that is invisible because the payloads are three-token strings and every receive is logged with `admitted: false` and `admitted_tokens: 0`, so no forwarded content entered any rank's context. In a real harness the forwarded blocks are somebody else's work product transiting a rank that has no business reading it. AgentMPI's answer is transport mode — forward the handle, not the body — and the mechanism exists and is used elsewhere in the repository (all fourteen `gather`

sends in `real-sw-p8-full` were `rendezvous`). But this sweep cannot demonstrate it, because the benchmark launched with `eager_limit=10**9`, which forces `Mode.AUTO` to choose eager for every payload. A harness adopting `bruck` for large agent artefacts should pin `mode=RENDEZVOUS` rather than rely on `AUTO`, and should note that the 2.58-fold volume penalty is then paid in handles rather than in context.

Against `linear` the case is unambiguous: fewer messages, fewer simultaneous unexpected arrivals, less blocking, and a shorter span, at the cost of $\log_2 p$ rounds instead of one. The only axis on which `linear` wins is the round count, and the cost model's selector — which returns `linear` for every `alltoall` at every size, because `predict` charges nothing per message for non-reduction ops — is the only thing in the repository that recommends it.

7 Threats to this reading

No agents, so no statement about model behaviour. The runs record no executor, zero agent calls, and 0.014–0.302 s wall times that are SQLite writes and event-log appends. The 55-fold blocking advantage over `linear` is a measurement of this fabric under thread contention. It happens to point the same way the agent cost model does, but the two are independent arguments and neither validates the other.

The volume discrepancy is arithmetic I performed, not something the tooling flagged. The exact counts in the table above come from evaluating $\sum_k |\{i < p : i \wedge 2^k\}|$ over the measured grid and comparing with the `tokens_sent` field summed over each run's `coll.alltoall` records. If the collective's self-reported token count were itself wrong in the same direction, the agreement I report would be two errors cancelling. What argues against that is that the two are computed by different code from different data — the self-report sums `_tok()` over the items actually placed in each message, while my figure is a pure combinatorial count — and they agree to the token at all thirteen distinct sizes.

Volume is measured in payload tokens, not on-wire tokens. The fabric's own `msg.send` records report a larger figure than the collective's self-report — 20,640 against 7,680 at $p = 32$ — because the fabric tokenises the whole `{"idx": [...], "vals": [...]}` envelope while the collective counts only the data items. With a three-token payload the envelope dominates, so the 2.7-fold gap is an artefact of the benchmark's tiny payloads and would shrink towards nothing with realistic artefacts. I have used the payload figure throughout for comparability with the closed form; a reader interested in what actually crosses the fabric should use the larger one and remember it is payload-size-dependent.

The round count is asserted by the implementation, not independently observed. `tr.stats.rounds = n_rounds` is set from the same $\lceil \log_2 p \rceil$ the closed form uses, so the agreement in the `rounds` column is a consistency check between two expressions of one intent rather than a measurement. The message tags do carry the round index, and grouping the $p = 32$ sends by it gives exactly five groups of 32, which is independent evidence for both the round count and the one-exchange-per-rank-per-round structure.